0096

National University of Computer and Emerging Sciences
Islamabad Campus

Spelling = Supose in Q#2.

Data structures (CS2001)

Sessional-II Exam

Date: November 8th 2025

Course Instructor(s)

Ms. Marium Hida, Mr. Aqib Rehman, Ms. Momina Behzad, Dr. Muhammad Rizwan

Section(s): BS-CS 24 (A,B,C,D,E,F,G)

Total Time (Hrs):	1
Total Marks:	75
Total Questions:	5

Invigilator's Signature

24i-0647 Roll No

D Section

Student Signature

Attempt all the questions.

Question No	Q1	Q2	Q3	Q4	Q5	Total
Marks Obt.	03	08	7	25	12	53
Max Marks	06	16	16	25	12	75

+5

Attempt Q 1,2,3 and 5 on Question paper and Q 4 on answer book.

In Q#2 Suppose is written as Supoe so that is spelling mistake.

CLO-1: Demonstrate basic concepts of data structure and algorithms.

Question 1: In a group project activity, n students are standing in a circle in their classroom. To decide the presentation order, the teacher gives a sheet of paper to the first student (A). Each student writes their name and project topic on the paper before passing it to the next student clockwise. This continues until the paper returns back to A, completing one full round. Your task is to find out how many times the paper changes hands before returning to the first student. You need to provide recursive function to solve this problem. **[6 marks]**

Use your understanding of recursion and complete the following code.

```
#include <iostream>
using namespace std;
int countPaperPasses(int n) {
    // Base case:
    if( n == 0 )
        return 0;
    if( n==1 )
        return 1;
    // Recursive case:
    return countPaperPasses(n-1) + 1;
```

```
1. int main() {
2. int n;
3. cout << "Enter number of students standing in circle: ";
4. cin >> n;
5. int passes = countPaperPasses(n);
6. cout << "The paper returns to the first student after " << passes << " passes.\n";
7. return 0;
8. }
```

CLO-4: Determine bugs in programs and recognize required operations with data structures

Question 2: Supose that Stack is implemented with push, pop and show method, Queue is implemented with enqueue, dequeue and show method. Both data structures are implemented using linked list. Trace the following code and identify the required output and show as per demand of question. **[16 marks]**

```
1. void f1(Stack& s) {
2. Stack t;
3. while (!s.empty()) {
4. int x = s.pop();
5. while (!t.empty() && t.top() > x)
   s.push(t.pop());
6. t.push(x);
7. }
8. while (!t.empty()) s.push(t.pop());
9. }
10. void f2(Queue& q) {
11. Stack s;
12. while (!q.empty()){
13. s.push(q.dequeue());
14. }
15. while (!s.empty()){
16. q.enqueue(s.pop());
17. }
18. }
19. void f3(Queue& q) {
20. Stack s;
21. int size = 0;
22. Queue temp;
23. while (!q.empty()) {
24. temp.enqueue(q.dequeue()); size++;
25. }
26. for (int i = 0; i < size; i++) {
27. int minVal = 9999;
28. for (int j = 0; j < size; j++) {
29. int val = temp.dequeue();
30. if (j < size - i && val < minVal) minVal =
    val;
31. temp.enqueue(val);
32. }
33. for (int j = 0; j < size; j++) {
35. int val = temp.dequeue();
36. if (val == minVal && j < size - i)
    s.push(val);
37. else temp.enqueue(val);
38. }
39. }
40. while (!s.empty()) q.enqueue(s.pop());
41. }
42. int main() {
43. Stack s;
44. Queue q;
45. s.push(3);
46. s.push(8);
47. s.push(1);
48. s.push(5);
49. s.push(2);
50. q.enqueue(9);
51. q.enqueue(4);
52. q.enqueue(6);
53. q.enqueue(2);
54. q.enqueue(7);
55. s.show();
56. q.show();
57. f1(s);
58. s.show();
59. q.show();
60. f2(q);
61. s.show();
62. q.show();
63. f3(q);
64. s.show();
65. q.show();
66. return 0;}
```

After F1: (2+2 marks)
Stack: (top -> bottom)

2	5	1	8	3

Queue: (front -> Rear)

9	4	6	2	7

After F1: (2+2 marks)
Stack : (top -> bottom)

8	5	3	2	1

Queue: (front -> Rear)

9	4	6	2	7

After F2: (2+2 marks)
Stack: (top -> bottom)

8	5	3	2	1

Queue: (front -> Rear)

7	2	6	4	9

After F3: (2+2 marks)
Stack: (top -> bottom)

8	5	3	2	1

Queue: (front -> Rear)

2	4	6	7	9

CLO-4: Determine bugs in programs and recognize required operations with data structures

Question 3: You are given the following C++ program. Each iteration performs a **different operation** that changes the tree structure. **Your Tasks:** **[16 marks]**

i. Examine the code **iteration by iteration**.
ii. For each iteration, record, entries in a table

```
1.  #include <iostream>

2.  struct Node {
3.  int key;
4.  Node *l, *r;
5.  Node(int k) : key(k), l(nullptr), r(nullptr)
    {}
6.  };

7.  Node* Foo1(int k){ return new Node(k); }

8.  Node* Foo2(Node* root, int k){
9.  if(!root) return Foo1(k);
10. if(k < root->key) root->l = Foo2(root->l,
    k);
11. else if(k > root->key) root->r = Foo2(root-
    >r, k);
12. return root;
13. }

14. Node* Foo3(Node* r){ while(r && r->l)
    r=r->l; return r; }

15. Node* Foo4(Node* root, int k){
16. if(!root) return nullptr;
17. if(k < root->key) root->l = Foo4(root->l,
    k);
18. else if(k > root->key) root->r = Foo4(root-
    >r, k);
19. else{
20. if(!root->l){ Node* t=root->r; delete root;
    return t; }
21. if(!root->r){ Node* t=root->l; delete root;
    return t; }
22. Node* m = Foo3(root->r);
```

```
23. root->key = m->key;
24. root->r = Foo4(root->r, m->key);
25. }
26. return root;
27. }

28. Node* Foo5(Node* x){
29. if(!x || !x->r) return x;
30. Node* y = x->r; x->r = y->l; y->l = x;
    return y;
31. }

32. Node* Foo6(Node* x){
33. if(!x || !x->l) return x;
34. Node* y = x->l; x->l = y->r; y->r = x;
    return y;
35. }

36. Node* Foo7(Node* root, int key, char
    dir){
37. if(!root) return nullptr;
38. if(key < root->key) root->l = Foo7(root->l,
    key, dir);
39. else if(key > root->key) root->r =
    Foo7(root->r, key, dir);
40. else{
41. if(dir=='L') root = Foo5(root);
42. else        root = Foo6(root);
43. }
44. return root;
45. }

46. Node* Foo8(Node* root, int key){
47. if(!root) return nullptr;
48. if(key < root->key) root->l = Foo8(root->l,
    key);
49. else if(key > root->key) root->r =
    Foo8(root->r, key);
50. else{ Node* t=root->l; root->l=root->r;
    root->r=t; }
51. return root;
52. }

53. Node* Foo9(Node* root){
54. if(!root) return nullptr;
55. Node* t = root->l; root->l = Foo9(root->r);
    root->r = Foo9(t);
56. return root;
57. }

58. Node* Foo10(Node* root, int key, int a, int
    b){
59. if(!root) return nullptr;
60. if(key < root->key) root->l = Foo10(root-
    >l, key, a, b);
61. else if(key > root->key) root->r =
    Foo10(root->r, key, a, b);
62. else{
63. Node* t = Foo1(a);
64. t->r = Foo1(b);
65. if(!root->l) root->l = t;
66. else if(!root->r) root->r = t;
67. else {
68. Node* cur = root->r;
69. while(cur->l) cur=cur->l;
70. cur->l = t;
71. }
72. }
73. return root;
74. }

75. void Foo11(Node* r){
76. if(!r) return;
77. Foo11(r->l);
78. Foo11(r->r);
79. delete r;
80. }

81. Node* Foo12(Node* root, int key){
82. if(!root) return nullptr;
83. if(key < root->key) root->l = Foo12(root-
    >l, key);
84. else if(key > root->key) root->r =
    Foo12(root->r, key);
85. else{ Foo11(root); return nullptr; }
86. return root;
87. }

88. int Foo13(Node* r){ return r? 1+Foo13(r-
    >l)+Foo13(r->r):0; }

89. void Foo14(Node* r, int depth){
90. if(!r) return;
91. Foo14(r->r, depth+1);
```

```
92. for(int i=0;i<depth;i++) std::cout << "    ";
93. std::cout << r->key << "\n";
94. Foo14(r->l, depth+1);
95. }

96. struct Step { char op; int x, y; };
97. int main(){
98. Node* root = nullptr;
99. int seed[] = {7,3,9,1,5,8,10};
100.        for (int v : seed) root = Foo2(root,
    v);
101.        Step steps[] = {
102.        {'A', 6, 0},
103.        {'C', 3, 0},
104.        {'E', 7, 0},
105.        {'B', 9, 0},
106.        {'D', 7, 0},
107.        {'F', 0, 0},
108.        {'G', 5, 2},
109.        {'H', 5, 0}
110.        };
111.        int N = 3
    static_cast<int>(sizeof(steps)/sizeof(steps[
    0]));

112.        for(int it=1; it<=N; ++it){
113.        char op = steps[it-1].op; int
    x=steps[it-1].x, y=steps[it-1].y;
114.        std::cout << "---------- Iteration "
    << it << " ----------\n";
115.        switch(op){
116.        case 'A': root = Foo2(root, x);
    std::cout << "Operation A on " << x <<
    "\n"; break;
117.        case 'B': root = Foo4(root, x);
    std::cout << "Operation B on " << x <<
    "\n"; break;
118.        case 'C': root = Foo7(root, x, 'L');
    std::cout << "Operation C on " << x <<
    "\n"; break;
119.        case 'D': root = Foo7(root, x, 'R');
    std::cout << "Operation D on " << x <<
    "\n"; break;
120.        case 'E': root = Foo8(root, x);
    std::cout << "Operation E on " << x <<
    "\n"; break;
121.        case 'F': root = Foo9(root);
    std::cout << "Operation F\n"; break;
122.        case 'G': root = Foo10(root, x, x,
    x+4); std::cout << "Operation G on " << x
    << "\n"; break;
123.        case 'H': root = Foo12(root, x);
    std::cout << "Operation H on " << x <<
    "\n"; break;
124.        }
125.        std::cout << "Current Tree:\n";
126.        Foo14(root, 0);
127.        std::cout << "Node Count: " <<
    Foo13(root) << "\n\n";
128.        }

129.        return 0;
130.        }
```

[2 each]

Table to Fill After Each Iteration

Iteration	Operation	Operation Type	Root Key	Node Count	In-order Traversal
Sample	Case 'X' on node value '*'	E.g. Leaf node count	Value of root node	No. of nodes in tree	Comma separated in order traversal
1	'A'	make BST	10	1	10
2	'C'	delete insert	7	3	3, 7, 10
3	'E'	attach delete insert	7	4	3, 7, 9, 10
4	'B'	deletion of node	7	4	3, 7, 9, 10
5	'D'	attach right/left	7	5	3, 7, 7, 9, 10
6	'F'	returns minimum	7	5	3, 7, 8, 9, 10
7	'G'	insert & balance	7	7	5, 8, 7, 7, 9, 10
8	'H'	insert & balan	7	7	5, 5, 6, 7, 7, 9, 10

CLO-3: Design appropriate data structures to solve real world problems related to the program

Question 4: Construct an AVL Tree step by step as per following instructions **[25 marks]**

i. Insert the following values into AVL tree 51,52,53,54,55,56,57,58,59,60,62,61,63,64,65
ii. At each insertion, if any rotation is to be applied, identify the imbalanced node and its balancing factor. Redraw the tree at each transformation.
iii. At any point if the tree is Full tree/ Complete tree / Almost complete binary tree mention it on the right side of the tree
iv. For 5 bonus marks, there is one spelling mistake in the paper find it and write it on top right corner of your question paper.

CLO-3: Design appropriate data structures to solve real world problems related to the program

Question 5: Write a recursive code in C++ that checks whether any given binary tree is a Binary search tree or not. Use following function signature to write the code. **[12 marks]**

Write code here: ***bool isBST(Node* root, int minValue, int maxValue)***

```
if (root == nullptr) {
    return true;
}
if (root -> value < minValue || root -> value >
    maxValue) {
    return false;
}
bool x = isBST(root -> left, minValue, root -> value);
bool y = isBST(root -> right, root -> value, maxValue);
if (x == false || y == false) { return false; };
return true;
```